

Civic Services Surveillance & Monitoring

Model Training Results Comparison and Dataset Improvement Report

2 September 2026

Abstract

This report compares two YOLO11n retraining runs of the 7-class civic-issue detector (`civic_services_unbalanced` vs. `civic_services_oversampled`), both trained on the current `data/re-training_data` pool with identical hyperparameters, differing only in whether minority classes were oversampled before training. It reports overall and per-class metrics, an overfitting analysis from the epoch-by-epoch loss curves, and a real annotation-quality defect found and partially fixed in the `billboard_hoarding` class. It also documents the extra data-extraction and review-tooling work under way to keep improving class coverage and label quality: a live review/annotation dashboard for newly sourced candidate images, a class-balancing script, and a purpose-built format-triage dashboard for the billboard annotation defect.

Contents

1	Training Setup	1
2	Overall Results Comparison	2
2.1	Per-Class AP50	2
2.2	Confusion Matrices	3
2.3	Precision-Recall Curves	3
3	Overfitting Analysis	3
4	Qualitative Detection Samples	4
5	Extra Work: Data Extraction & Class-Quality Improvement	5
5.1	Review & Annotation Dashboard (<code>review_annotate_dashboard.py</code>)	5
5.2	Class Balancing (<code>balance_train_data.py</code>)	5
5.3	Billboard Annotation-Format Defect — Found and Partially Fixed	6
6	Summary & Recommendations	6

1 Training Setup

Both runs used identical hyperparameters (YOLO11n, 100 epochs, batch 32, image size 640, `copy_paste=0.1`, `cos_lr=True`), trained against `configs/data.yaml` → `data/re-training_data`. The only difference is the training-set composition:

- **unbalanced** — natural per-class distribution of `data/re-training_data`'s train split.
- **oversampled** — `balance_train_data.py` first duplicates minority-class train images/labels (round-robin) up to the largest class's count; val/test splits are left untouched in both runs, so evaluation is on identical, non-duplicated data either way.

2 Overall Results Comparison

Table 1: Overall validation metrics, re-evaluated from each run’s saved `best.pt` against the same val split.

Run	mAP50	mAP50-95	Precision	Recall
unbalanced	0.703	0.452	0.788	0.624
oversampled	0.689	0.457	0.764	0.681

Oversampling raises recall by **5.7 points** and mAP50-95 (the localization-strict metric) slightly, at a cost of 1.4 points of mAP50 and 2.4 points of precision. This is a genuine trade-off, not a clean win either way — see Section 2.1 for which classes actually drove this.

2.1 Per-Class AP50

Table 2: Per-class AP50 and validation support (val split only has 145 images / 209 instances total — several classes have single-digit image counts, see note below).

Class	unbalanced	oversampled	Δ	val imgs / inst.
garbage_pile	0.277	0.236	−0.041	13 / 29
pothole	0.663	0.716	+0.053	12 / 22
manhole_open_broken	0.810	0.845	+0.035	27 / 29
streetlight_pole	0.599	0.477	−0.122	73 / 87
encroachment	0.959	0.995	+0.036	6 / 16
billboard_hoarding	0.921	0.891	−0.030	10 / 30
cattle	0.691	0.664	−0.027	4 / 13

Key observations:

- `pothole` and `manhole_open_broken` — the two classes the `copy_paste` augmentation specifically targets — both improved under oversampling, as intended.
- `streetlight_pole`, the *largest* class in the training pool (~ 727 images), regressed by 12.2 points under oversampling despite being large — plausible explanation: duplicating minority classes diluted its relative frequency per epoch.
- `garbage_pile` is weak in *both* runs (0.24–0.28), well below its historical AP50 of ~ 0.61 –0.65 on the earlier, smaller dataset — this does not match expectations and needs investigation (label-quality check on the newly added `data/new_review_data/garbage_pile` images is the leading suspect).
- `cattle` and `encroachment` have only 4 and 6 val images respectively — their AP50 numbers are not statistically reliable in either direction.

2.2 Confusion Matrices

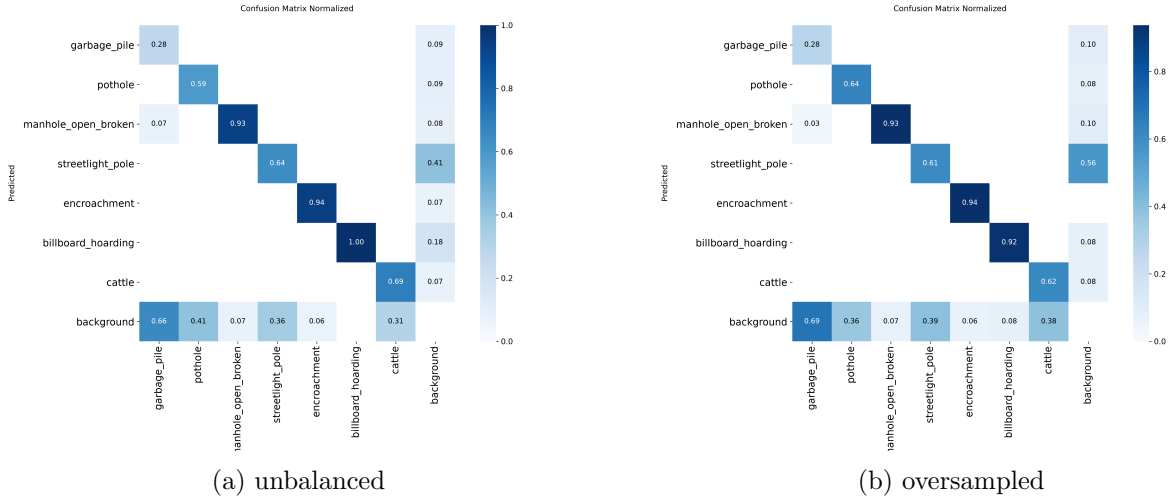


Figure 1: Normalized confusion matrices (rows = true class, background column captures missed/false detections). `garbage_pile`'s weak diagonal in both matrices is the clearest visual confirmation of the AP50 weakness noted above.

2.3 Precision-Recall Curves

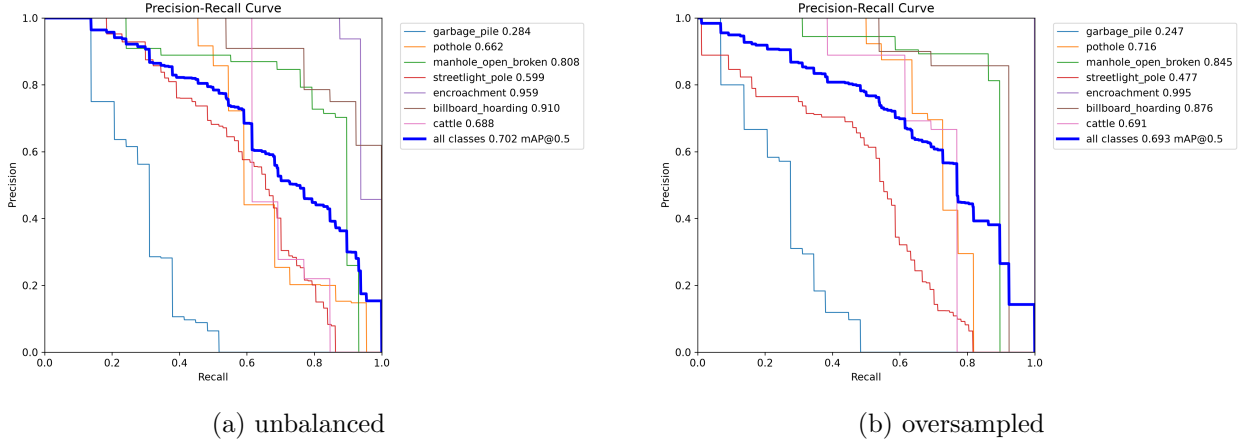


Figure 2: Per-class precision-recall curves. Note `streetlight_pole`'s (orange/relevant curve) visibly lower area under oversampling, consistent with its AP50 regression.

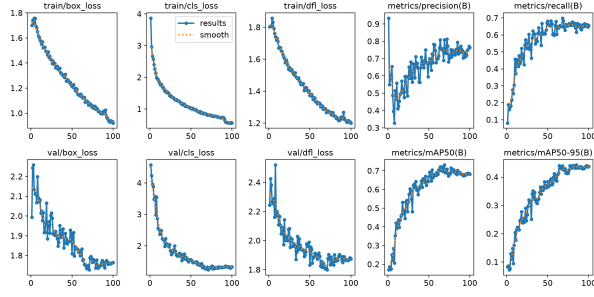
3 Overfitting Analysis

Both runs show the classic overfitting signature: **training loss keeps falling for all 100 epochs while validation loss plateaus around epoch 60–80 and barely moves after that** — the train/val gap widens continuously instead of both curves flattening together.

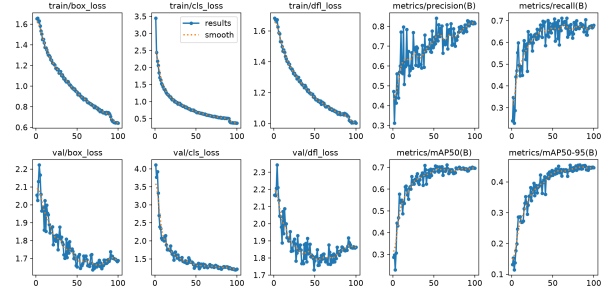
Table 3: Box-loss trend, selected epochs. Oversampled shows a larger absolute gap because duplicated train images let the model partially memorize repeats.

Run	epoch	train_box_loss	val_box_loss	gap
unbalanced	40	1.291	1.838	0.547
unbalanced	100	0.922	1.764	0.842
oversampled	40	1.025	1.731	0.706
oversampled	100	0.642	1.689	1.047

mAP50-95 peaks around epoch 83–85 in both runs and only wobbles afterward — the extra ~15–20 epochs of training past that point are not buying further generalization. `train.py`’s early-stopping `patience` is set to 20 and never triggered (each 20-epoch window still produced a marginal new best), so tightening it (e.g. to 10) would likely reach an equivalent result faster without changing outcomes, since Ultralytics already saves `best.pt` at the true best epoch rather than the final one.



(a) unbalanced



(b) oversampled

Figure 3: Full per-epoch loss/metric curves. In both, the box/cls/dfl loss panels show train (blue) diverging from val (orange) after roughly epoch 60–80.

4 Qualitative Detection Samples



(a) unbalanced — val batch predictions



(b) oversampled — val batch predictions

Figure 4: Model predictions on the same held-out validation batch for each run.

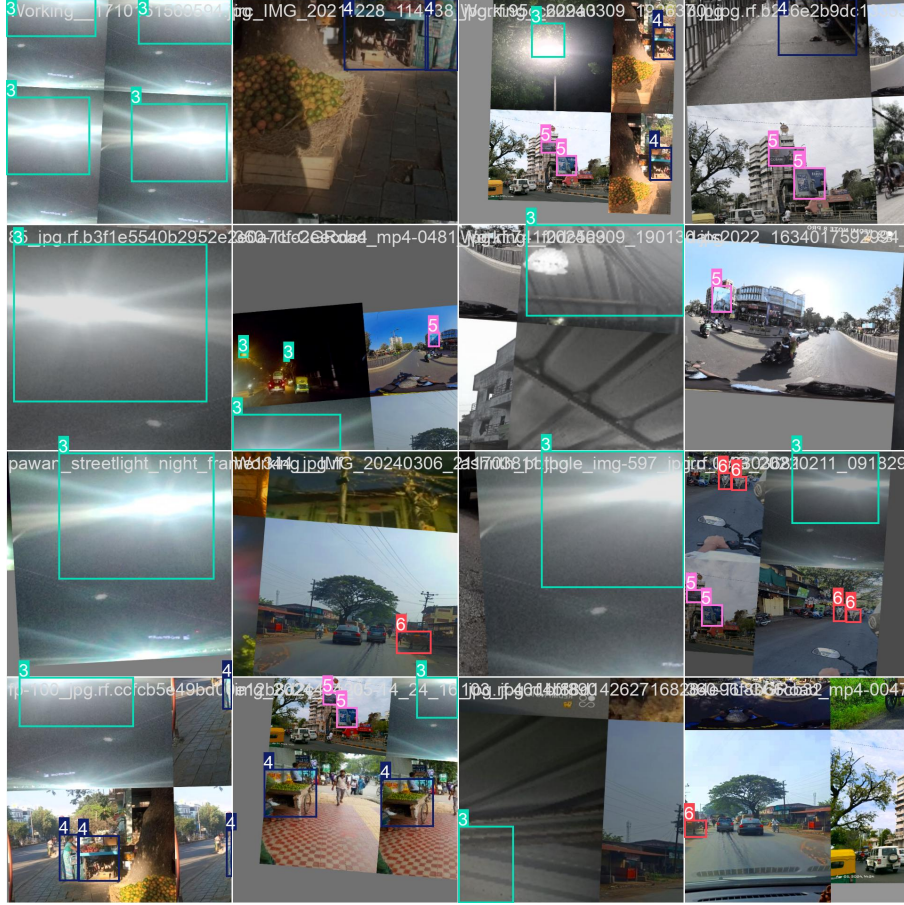


Figure 5: A training mosaic batch from the oversampled run, showing the augmentation pipeline in effect (mosaic, copy-paste, color jitter) — this is what the model actually sees per step, including duplicated minority-class crops pasted onto other scenes.

5 Extra Work: Data Extraction & Class-Quality Improvement

Alongside the two training runs above, ongoing work is targeted specifically at improving the weakest classes and fixing label-quality defects, rather than just adding more raw volume.

5.1 Review & Annotation Dashboard (`review_annotate_dashboard.py`)

A live, LAN-reachable Flask dashboard (port 5053) queues newly sourced candidate images per class from `data/new_review_data/` — these arrive with auto-remapped boxes from their source datasets, shown pre-drawn and editable rather than annotated from scratch. It also surfaces the 17 legitimate empty-label `streetlight_pole` negatives already inside `data/re-training_data`. A dedicated `_background_negatives` queue was added to start building genuine no-object-of-interest training samples, directly targeting the model’s long-standing lack of a background class (a confirmed source of confident false positives on out-of-distribution input in earlier testing). Reviewer decisions persist in `data/annotation_review_progress.json` so the queue only ever shows what is still pending.

5.2 Class Balancing (`balance_train_data.py`)

Round-robin duplicates minority-class train images/labels up to the current largest class’s count, writing to both the flat `data/re-training_data` tree (what `train.py` reads) and the per-class `data/classwise_data` mirror so the two stay in sync. Val/test are never touched, keeping evaluation on genuine, non-duplicated data regardless of how training is balanced. This is what produced the `civic_services_oversampled` run analyzed above.

5.3 Billboard Annotation-Format Defect — Found and Partially Fixed

While comparing the two runs, 21 `billboard_hoarding` label files were found mixing two incompatible annotation formats: plain YOLO boxes (`class cx cy w h`, 5 fields) and segmentation polygons (`class x1 y1 x2 y2 ...`, an odd field count > 5). Ultralytics silently drops any file mixing both formats during train/val ("`labels mix segment and detection rows`"), so these images were contributing nothing. All 21 affected files traced back to a single source, `pawan_billboard` — the same source separately found to be $\sim 29\%$ non-Indian stock photography (Bangkok, Buenos Aires, Rayong, Chonburi) by filename inspection, and which supplied *100% of the val and test billboard_hoarding images* before this fix, meaning the class’s reported AP50 (0.89–0.92 above) was being measured on an unrepresentative, partly-foreign held-out set.

A purpose-built dashboard, `billboard_format_review.py` (port 5054), was written to triage this: it classifies every `billboard_hoarding` label file into *YOLO box (clean)*, *segmentation only*, or *mixed*, renders polygons and their derived bounding boxes for visual comparison, and lets a reviewer convert, edit, or reject each one. The dashboard **live-rescans** the dataset on a short TTL (rather than a one-time snapshot at startup) and polls for changes every 4 seconds, so it reflects edits made directly on disk — e.g. by an external cleanup script — without needing a restart.

All 21 affected files were removed directly from `data/re-training_data` and `data/classwise_data` (image + label pairs, both locations kept in sync). `billboard_hoarding` counts after cleanup:

Table 4: billboard_hoarding counts before/after removing the 21 format-defective files.

Split	before	after	removed
train	279	262	17
val	10	6	4
test	10	10	0

Not yet done: the val/test split is still not re-diversified across sources (val is currently the remaining `pawan_billboard` images only) — a proper re-shuffle across all `billboard_hoarding` sources is needed before this class’s metrics can be trusted, and a retrain is needed to reflect the cleaned label set.

6 Summary & Recommendations

- **No clean winner between the two runs** — oversampled trades mAP50/precision for recall and helps the two classes `copy_paste` targets, but regresses the largest class (`streetlight_pole`). Neither should be deployed as-is without addressing `garbage_pile`’s weakness and the `billboard_hoarding` split issue first.
- **Overfitting is present in both**, more so in oversampled due to image duplication; `best.pt` already mitigates this by selecting the best epoch, but tightening `patience` would save training time with no expected quality loss.
- **garbage_pile needs investigation** — its AP50 collapsed relative to historical numbers on the older dataset; check the newly added `data/new_review_data/garbage_pile` images for label-quality issues.
- **Billboard val/test split needs re-shuffling** across all sources, not just `pawan_billboard`, before its metrics can be trusted; a retrain is needed after that to pick up the 21 removed files’ absence and any re-split changes.